

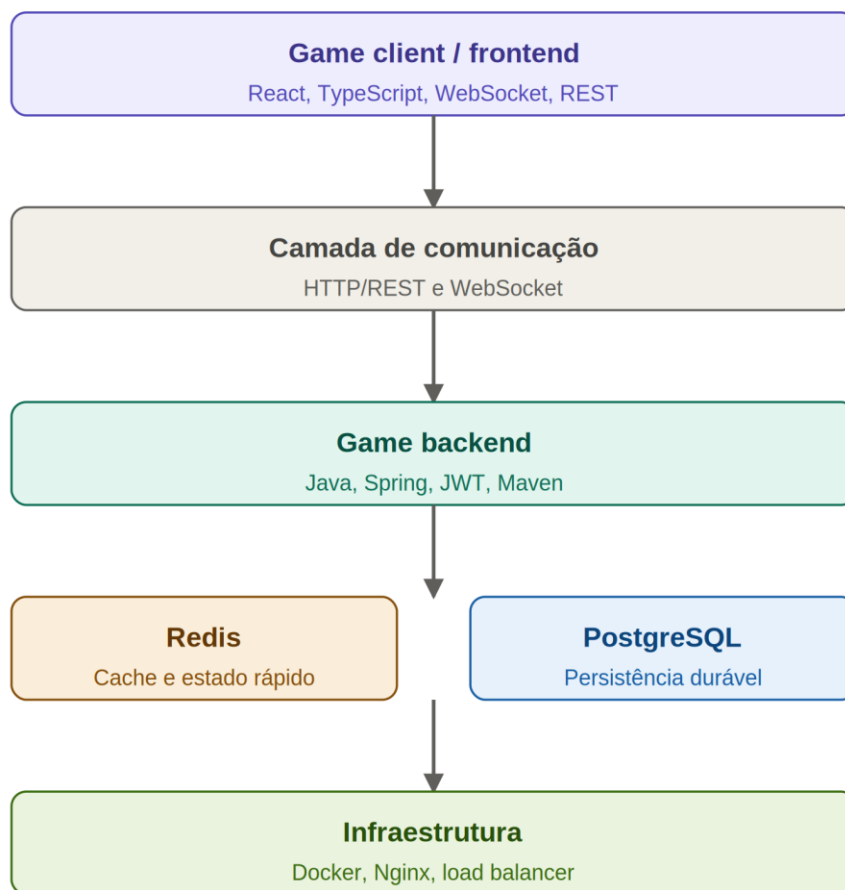
Arquitetura de Software

Jogo multiplayer por turnos

Documentação técnica da arquitetura de um jogo multiplayer baseado em turnos, cobrindo cliente, backend, comunicação em tempo real, persistência e infraestrutura.

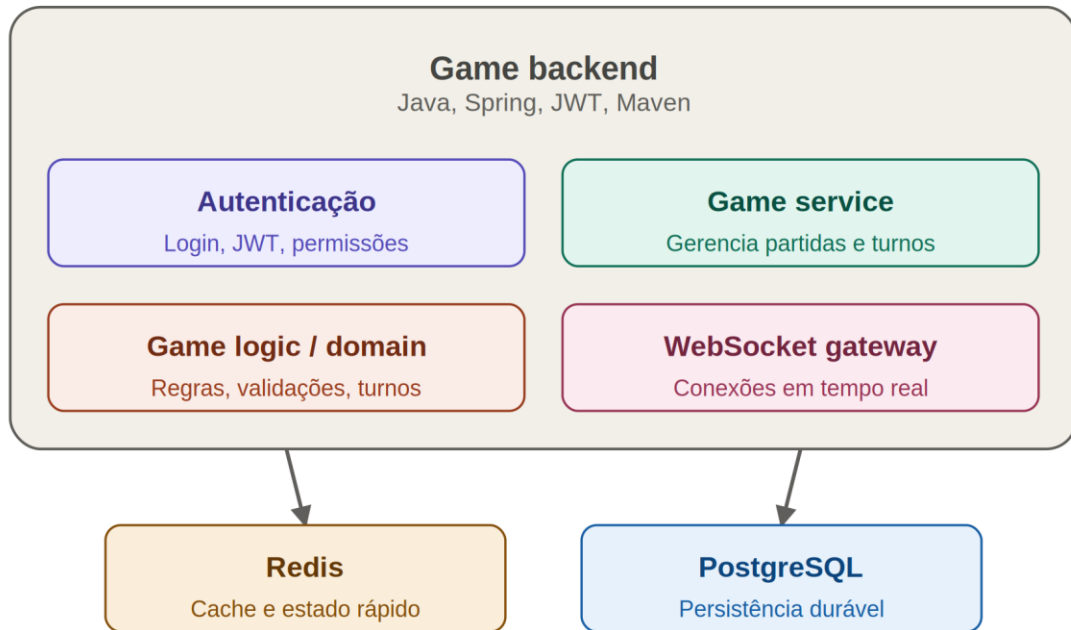
1. Visão geral em camadas

O cliente cuida de renderização e entrada do jogador; a camada de comunicação combina REST e WebSocket rodando no mesmo Game Server; o backend concentra toda a lógica; e a persistência é dividida entre Redis (rápido, temporário) e PostgreSQL (permanente).



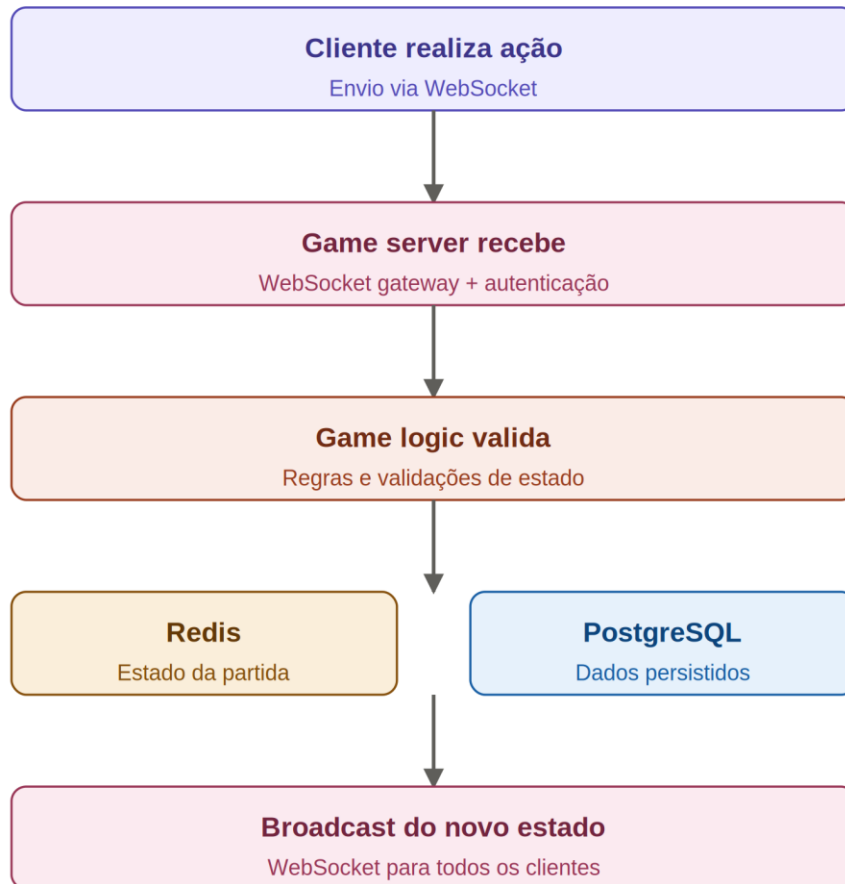
2. Componentes internos do Game Backend

A validação oficial de qualquer ação acontece sempre no Game Logic, no servidor. O WebSocket Gateway mantém as conexões mas não persiste nada por conta própria — quem acessa Redis e PostgreSQL é sempre o backend, nunca o cliente diretamente.



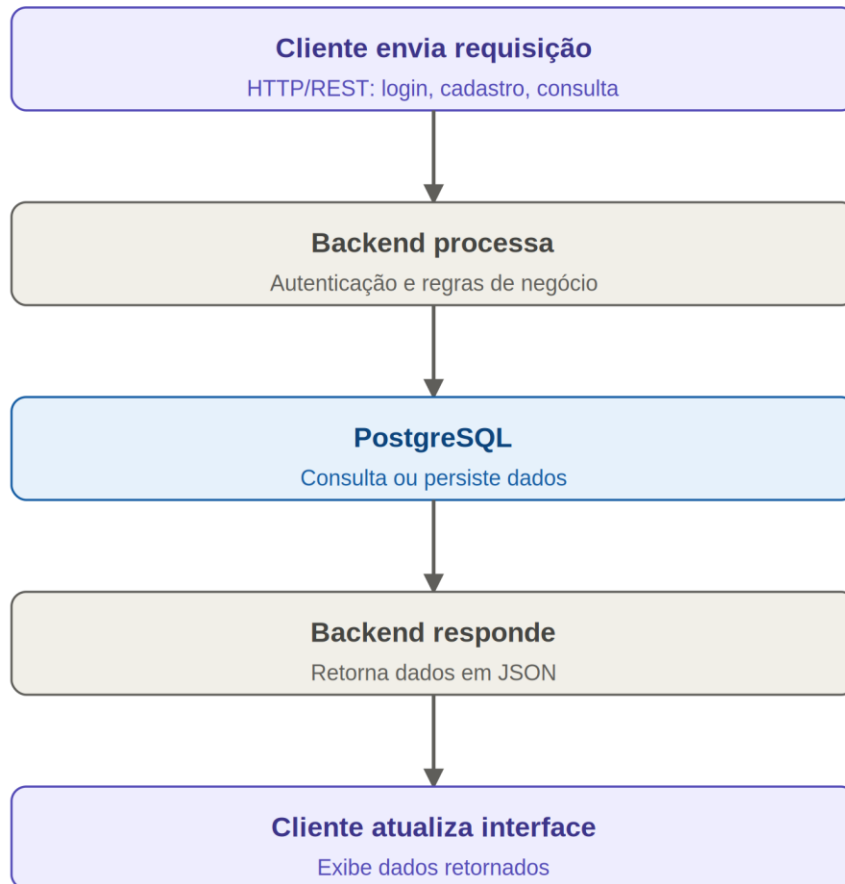
3. Fluxo principal de uma ação (WebSocket)

Sequência de uma ação dentro de uma partida ativa: o cliente envia a ação, o servidor autentica e valida, o estado é atualizado em Redis e/ou PostgreSQL, e o novo estado é enviado de volta a todos os clientes conectados.



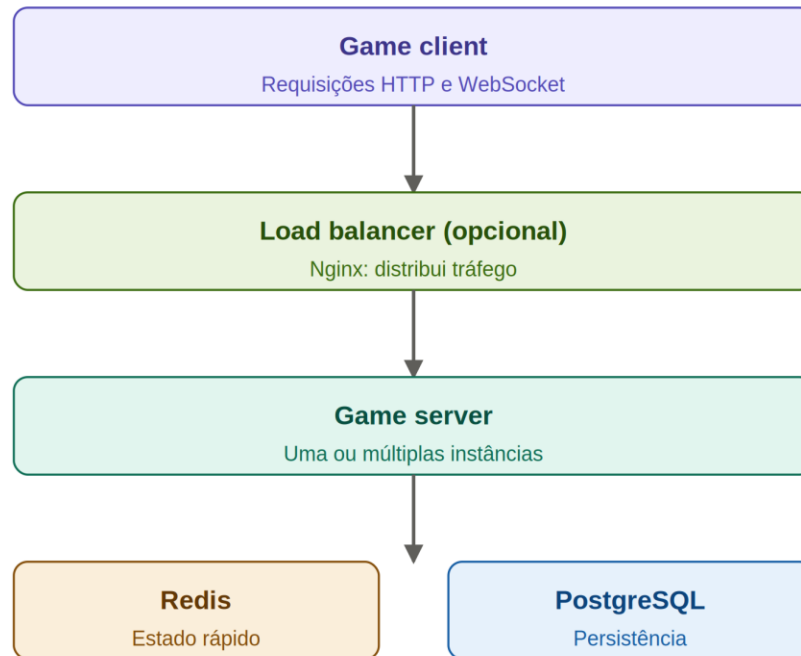
4. Fluxo HTTP/REST

Fluxo para operações que não exigem comunicação contínua em tempo real, como login, cadastro, perfil e consultas de histórico.



5. Evolução da infraestrutura

Inicialmente, um único Game Server pode ser responsável por autenticação, gerenciamento de partidas, WebSocket, regras e persistência. A arquitetura pode evoluir para múltiplas instâncias atrás de um load balancer (Nginx), com Redis e PostgreSQL compartilhados.



6. REST vs WebSocket

	HTTP/REST	WebSocket
Padrão de comunicação	Requisição → resposta	Conexão persistente e bidirecional
Usado para	Login, cadastro, perfil, histórico, consultas	Ações do jogador, eventos, mudanças de turno, notificações
Quando é aberto	A cada chamada	Uma vez, mantido durante toda a partida
Convive com o outro?	Sim — ambos rodam no mesmo Game Server	Sim

7. Responsabilidades, resumidas

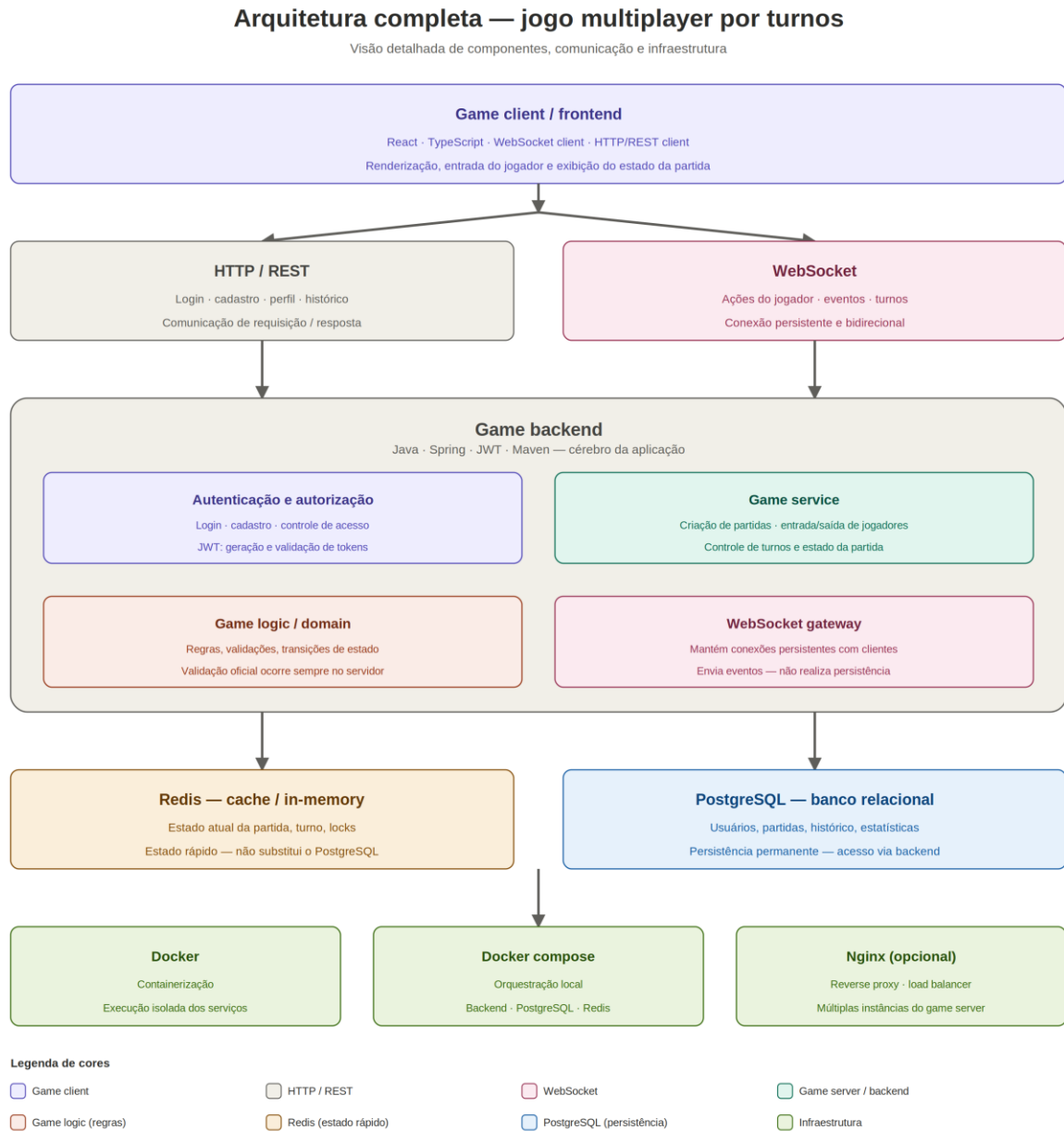
- Game server = cérebro da aplicação, coordena tudo
- Game logic = regras, validações e transições de estado (a validação oficial é sempre no servidor)
- WebSocket = comunicação em tempo real
- REST = comunicação tradicional request/response
- Redis = estado rápido, temporário — não substitui o PostgreSQL
- PostgreSQL = persistência durável, acessado apenas pelo backend
- Game client = interface, renderização e entrada do jogador

8. Tecnologias e função

Tecnologia	Função
React	Interface do cliente do jogo
TypeScript	Linguagem do cliente
Java	Linguagem do backend
Spring	Framework do backend
REST API	Comunicação tradicional
WebSocket	Comunicação bidirecional em tempo real
JWT	Autenticação e autorização
Maven	Build e gerência de dependências
Redis	Cache e estado rápido/temporário
PostgreSQL	Persistência relacional
Docker	Containerização
Docker Compose	Orquestração local (backend, PostgreSQL, Redis)
Nginx	Reverse proxy / load balancer

9. Arquitetura completa

Visão consolidada de todos os componentes descritos nas seções anteriores, em um único diagrama.



10. Estratégia de deploy

Para viabilizar o projeto como iniciativa acadêmica, o deploy foi planejado inteiramente sobre planos gratuitos ("free tier") de provedores de nuvem, sem custo de infraestrutura, tanto em ambiente de desenvolvimento quanto de produção.

10.1 Ambiente de desenvolvimento

O ambiente de desenvolvimento é executado localmente via Docker Compose, orquestrando três serviços: backend (Spring Boot), PostgreSQL e Redis. O frontend roda fora do Docker, diretamente com o servidor de desenvolvimento do Vite (npm run dev), apontando para o backend local (localhost:8080) através de variáveis de ambiente.

- docker compose up sobe backend + banco + cache localmente.
- O frontend consome a API via variável de ambiente (ex.: VITE_API_URL=http://localhost:8080).
- Após qualquer alteração no backend, basta docker compose up --build para reconstruir a imagem e refletir a mudança.

10.2 Ambiente de produção

Em produção, cada componente da arquitetura é hospedado em um provedor especializado, todos com camada gratuita persistente (sem prazo de expiração):

Componente	Serviço	Custo	Limites do plano gratuito
Frontend (React/TS)	Vercel	Gratuito	100GB de banda/mês, builds ilimitados para projetos pessoais, sem SLA de uptime
Backend (Spring Boot + WebSocket)	Render	Gratuito	750h de execução/mês; instância "dorme" após ~15 min de inatividade (cold start ao acordar ~1 min)
Banco de dados (PostgreSQL)	Neon	Gratuito	100 CU-horas de compute/mês por projeto, 0,5GB storage/projeto, compute escala a zero após 5 min
Cache/estado (Redis)	Upstash	Gratuito	500.000 comandos/mês, 256MB de dados

O frontend, hospedado na Vercel, consome o backend publicado no Render através de uma URL pública fixa, configurada via variável de ambiente no momento do build. A atualização do backend ocorre por deploy contínuo: cada push no repositório Git aciona a reconstrução automática da imagem Docker e a substituição da instância em produção, sem necessidade de qualquer ação do lado do frontend — desde que o contrato da API não seja alterado.

10.3 Por que não utilizar a AWS

A AWS foi avaliada como alternativa de deploy e também oferece uma camada gratuita (AWS Free Tier) capaz de hospedar toda a arquitetura:

Componente	Serviço AWS	Observação
Backend (Docker/Spring)	EC2 (t2.micro/t3.micro)	750h/mês grátis no 1º ano — dá pra rodar o docker-compose.yml inteiro numa única VM
PostgreSQL	RDS (free tier)	750h/mês, 20GB de storage, disponível apenas no 1º ano da conta
Redis	ElastiCache	Free tier limitado, também restrito ao 1º ano
Frontend estático	S3 + CloudFront	Hospedagem de arquivos estáticos, praticamente sempre gratuita para baixo tráfego
Load balancer	ALB	Não é gratuito — cobra por hora mesmo dentro do free tier

Apesar de tecnicamente viável, a AWS foi descartada para este projeto pelos seguintes motivos:

- Complexidade de configuração: exige configurar VPC, Security Groups, IAM e outros recursos antes mesmo de publicar a aplicação, diferente do fluxo "conectar repositório e publicar" oferecido por Render e Vercel.
- Free tier com prazo de validade: a maior parte dos serviços gratuitos da AWS (EC2, RDS, ElastiCache) vale apenas nos primeiros 12 meses da conta, enquanto Render, Vercel, Neon e Upstash oferecem camadas gratuitas permanentes (ainda que limitadas).
- Risco de custo inesperado: recursos como o Load Balancer (ALB) já são cobrados por hora mesmo dentro do free tier, e é fácil gerar cobranças não previstas (tráfego de saída, instância esquecida ligada), o que é um risco desnecessário para um projeto de estudante.
- Escopo do projeto: por ser um projeto acadêmico de baixo tráfego, a arquitetura não exige a escalabilidade ou os recursos avançados que justificariam a complexidade adicional da AWS.

A AWS voltaria a ser uma opção justificável caso a disciplina exigisse experiência prática com a plataforma, ou caso o time optasse por consolidar toda a stack (backend, banco e Redis) em uma única instância EC2 rodando o docker-compose.yml, aproveitando créditos do AWS Educate ou do GitHub Student Developer Pack para mitigar o risco de custo.